

Analisi del Report – Modulo 4 – MPI e MPI+OpenMP

Walkthrough completo del report, sezione per sezione

Partitioned Hash Join with Duplicates

SPM – Università di Pisa – Prof. M. Torquati – A.A. 2025-2026

Walkthrough completo del report, sezione per sezione dall'inizio alla fine, con la motivazione di ogni scelta, la derivazione di ogni numero, la lettura di ogni grafico/tabella, il confronto con baseline/modulo precedente, i *risultati misurati* dove eseguibili senza cluster, e una sezione finale di ulteriori esperimenti. La struttura segue l'ordine del report originale, così puoi presentarlo scorrendo il PDF mentre mostri il documento al prof.

Contents

Come usare questo documento	2
1 Introduction (sezione 1)	2
2 Distributed Algorithm (sezione 2)	2
2.1 Partitioning and Rank Assignment	2
2.2 Pipeline	2
2.3 Choice of the Redistribution Primitive	3
3 Validation (sezione 3)	3
4 Experimental Evaluation (sezione 4)	3
4.1 Strong Scaling	3
4.2 Weak Scaling	3
4.3 Phase Breakdown e Skewed Workload	4
5 Discussion: Comparison with previous modules (sezione 5)	4
6 Hybrid MPI+OpenMP (sezione 6)	4
7 Conclusions (sezione 7)	4
8 Ulteriori esperimenti da fare	5

Come usare questo documento

Segue l'ordine del report (Introduction → Distributed Algorithm → Validation → Experimental Evaluation → Discussion → Hybrid MPI+OpenMP → Conclusions).

1 Introduction (sezione 1)

Il report dichiara: il join va in memoria distribuita, l'algoritmo non cambia (R e S partizionate con la stessa hash, join locale, una collettiva combina i risultati); con i dati sparsi, il join locale compete con la comunicazione che porta le partizioni sullo stesso rank. Due implementazioni che riusano kernel e generatore: MPI puro (`Alltoallv`) e ibrido (1 rank/nodo, fasi locali OpenMP).

Come difenderlo all'orale

8 fasi cronometrate, non 7. `histogram_local`, `scatter_local`, `comm_sizes`, `comm_payload`, `histogram_post`, `scatter_post`, `join_local`, `reduce_final`. La prosa accorpa `histogram_post/scatter_post`; il codice (`struct PhaseTimingMPI`) le separa.

2 Distributed Algorithm (sezione 2)

2.1 Partitioning and Rank Assignment

Motivazione della scelta

Hash invariata; `dest=pid%R`, `lp=pid/R`; vincolo $P\%R==0$ (ogni rank lo stesso numero di partizioni). Lo scatter usa il remap `dest-major pid'=dest-P_per_rank+lp` ⇒ send buffer già contiguo per destinazione = zero-copy sul lato invio dell'`Alltoallv`.

2.2 Pipeline

Il report elenca le 8 fasi e dice che ogni rank genera *solo* la sua slice con skip-ahead di `splitmix64` (niente scatter iniziale), che ogni fase di comunicazione è preceduta da `MPI_Barrier`, e che i tempi sono ridotti col massimo tra i rank.

Motivazione della scelta

Skip-ahead. `state += GOLDEN` per chiamata ⇒ stato dopo k draw = `seed+GOLDEN·k`. Ogni rank genera la sua slice, ma la concatenazione è byte-per-byte uguale al generatore sequenziale ⇒ niente `Scatterv` iniziale, costo zero fuori dalla regione cronometrata, baseline onesta. **Barriera prima del timer:** senza, un rank lento farebbe aspettare i veloci *dentro* la collettiva, attribuendo alla fase un imbalance a monte. **Rank-max:** il breakdown riporta il rank più lento = percorso critico.

Da dove viene il numero (derivazione)

Displs dell'Alltoallv. Prima `MPI_Alltoall` di 1 int (ogni rank sa quanti record riceverà). Poi `send_displs[j]` = prefix-sum esclusivo dei counts = offset del primo elemento per il rank j , dove lo scatter dest-major l'ha messo. Tipo `MPI_UINT64_T` (Record = 8 B densi). R e S in due `Alltoallv`.

2.3 Choice of the Redistribution Primitive

Come difenderlo all'orale

Alltoallv bloccante vs Isend/Irecv (quantificato nel report). L'overlap nasconderebbe lo scambio dietro il lavoro post; ma il post è ~ 100 ms su ~ 1.3 s in puro ($\sim 7\%$) e ~ 120 ms su 0.61 s nell'ibrido ($\sim 19\%$): guadagno al più $7\text{--}19\%$, e spezzaresti la collettiva tunata in scambi per-partizione perdendone lo scheduling interno. Trade-off aperto, collettiva bloccante mantenuta.

3 Validation (sezione 3)

Il report: tripla (join_count, ck1, ck2) a taglie piccola/media/grande; per le piccole anche join naive $O(N_R N_S)$ indipendente dal kernel (se danno la stessa tripla, il kernel è quasi certamente corretto); confronto contro hashjoin_seq; rank range $\{1, 2, 4, 8, 32, 64, 128\}$. 46 configurazioni passano.

4 Experimental Evaluation (sezione 4)

Setup: $P = 256$, seed 42, max_key= $N_R/2$, 5 ripetizioni, mediana. Strong: $N_R = 50M$, $N_S = 100M$. Weak: per-rank $2M \times 4M$. Puro: 32 rank/nodo. Ibrido: 1 rank/nodo, OMP_NUM_THREADS=32. Baseline seq compilato -march=ivybridge su nodo dedicato (i compute node non hanno l'AVX2 del login).

4.1 Strong Scaling

Fase	N=1	N=2	N=4	N=8
scatter_local	0.130	0.065	0.033	0.017
comm_payload	0.092	0.293	1.35	0.532
join_local	0.239	0.121	0.063	0.032
totale	0.65	0.59	1.47	0.63

Table 1: MPI puro, per-fase [s], uniforme.

Come difenderlo all'orale

Ogni fase locale scala $7\text{--}8\times$; comm_payload anti-scala, picca a 4 nodi/128 rank (1.35 s, $\approx 92\%$ del totale). Speedup uniforme: ibrido $2.8 \rightarrow 12.0\times$; puro picco $7.6\times$ (2 nodi), collasso $3.0\times$ (4 nodi), recupero $7.1\times$ (8 nodi/256 rank). **Prova rank-vs-nodi:** (1) 128 rank su 8 nodi (16/nodo) $\rightarrow$ stesso scambio lento; (2) forzando l'algoritmo basic-linear $\rightarrow$ stabile ma lento. Costo $\propto$ rank, varianza $\propto$ algoritmo della libreria.

4.2 Weak Scaling

Da dove viene il numero (derivazione)

Hockney: ibrido 0.58 vs puro 0.21. $t(m) = \alpha + \beta m$. In all-to-all ogni rank posta $\mathcal{R} - 1$ messaggi di taglia $V/\mathcal{R}$ (V fisso): $t \approx (\mathcal{R} - 1)\alpha + \beta V$. βV costante; cresce $(\mathcal{R} - 1)\alpha$, lineare in $\mathcal{R}$. Ibrido $\mathcal{R} =$ nodi: bandwidth-bound. Puro $\mathcal{R} = 32 \times$ nodi: start-up-bound. Conferma: con 1 rank/nodo lo scambio mediano sale da 15 a 52 ms ($1 \rightarrow 8$ nodi); con 32/nodo da 0.12 a 3.17 s ($32 \rightarrow 256$ rank).

4.3 Phase Breakdown e Skewed Workload

Da dove viene il numero (derivazione)

Baseline skewed (2.30 s) più veloce dell’uniforme (4.48 s). Per località: 90% dei record in 4 partizioni $\Rightarrow$ scatter in pochi stream quasi-contigui (cala $\sim 3.4\times$ nel breakdown seq) e tabelle calde cache-resident. Lo skew è “buono” per il seq, “cattivo” per il distribuito.

Come difenderlo all’orale

Il floor distribuito. Skew: puro $2.2 \rightarrow 1.4\times$, ibrido $1.5 \rightarrow 2.5\times$. Le 4 calde pinnate a pochi rank in memoria privata: nessun rank le cede senza muovere dati. Partitioned-state farm con hashing: l’hash bilancia il *numero* di partizioni, non il *volume*. In M3 **dynamic** ribilanciava; qui no.

5 Discussion: Comparison with previous modules (sezione 5)

Uniforme (seq 4.48 s)			Skewed (seq 2.30 s)		
Impl.	s	<i>S</i>	Impl.	s	<i>S</i>
M2 (32t,1n)	0.53	$8.4\times$	M3 loop (16t)	0.37	$6.2\times$
M3 loop (32t)	0.44	$10.1\times$	M4 puro (32r)	1.03	$2.2\times$
M4 ibr.(8n)	0.37	$12.0\times$	M4 ibr.(8n)	0.90	$2.5\times$

Come difenderlo all’orale

Cosa dimostra. Uniforme: 8 nodi ($12.0\times = 4.48/0.37$) battono di poco M3 su 1 nodo ($10.1\times = 4.48/0.44$) – $8\times$ l’hardware per $\sim 1.2\times$, perché la collettiva non si riduce coi nodi mentre il join locale sì (Amdahl: la collettiva è la frazione non-scalabile). Skew: il divario si *inverte*, M3 ($6.2\times$) batte M4 ibrido ($2.5\times$). **La vera giustificazione del distribuito è la capacità**, non la velocità; questo workload da ~ 1.2 GB non la esercita.

6 Hybrid MPI+OpenMP (sezione 6)

Motivazione della scelta

1 rank/nodo, MPI_THREAD_FUNNELED (solo il main chiama MPI, fuori dalle regioni OpenMP $\Rightarrow$ FUNNELED basta; MULTIPLE aggiungerebbe lock interni inutili). *Ogni* fase locale è OpenMP. **Due numeri chiave:** (1) parallelizzare *tutte* le fasi locali porta la prima versione solo-join da 6.25 s (peggio del seq, Amdahl-bound) a 1.61 s ($3.9\times$); (2) ridurre il fan-out da 256 a 8 rank porta l’Alltoallv da 0.53 a 0.18 s ($3\times$). L’Allreduce resta <1 ms.

7 Conclusions (sezione 7)

Il report chiude: entro 8 nodi l’ibrido raggiunge $12.0\times$ (WSE 0.58) contro $7.6\times/0.21$ del puro; il prossimo collo è la comunicazione (Alltoallv); le gains sono reali ma bounded – su uniforme 8 nodi $\approx$ M3 su 1, su skew M3 resta avanti; la vera giustificazione del distribuito è la capacità, che questo workload non esercita.

8 Ulteriori esperimenti da fare

Esperimento di supporto

1. Rank vs nodi

Come: 128 rank su 4 nodi e su 8 nodi (16/nodo); `probe_alltoallv.sh`

Misura: tempo e varianza di `comm_payload`

Atteso: stesso scambio lento in entrambi

Dimostra: che il collo e' il numero di rank, non di nodi.

2. Algoritmo della collettiva

Come: `--mca coll_tuned_alltoallv_algorithm basic-linear`

Misura: tempo e varianza

Atteso: stabile ma lento

Dimostra: che la varianza viene dall'algoritmo, il costo dal fan-out.

3. Ibrido 1 vs 2 rank/nodo

Come: 1×32 thread vs 2×16 (1/socket)

Misura: Alltoallv e join locale

Atteso: 1/nodo collettiva piu' economica; 2/nodo meno contesa NUMA

Dimostra: il trade-off fan-out vs NUMA.

4. Overlap Isend/Irecv (prototipo)

Come: avvia il join sulle partizioni gia' arrivate

Misura: tempo a 4 nodi

Atteso: guadagno $\leq 7-19\%$

Dimostra: empiricamente perche' non valeva la complessita'.

5. Remap skew-aware (prototipo)

Come: replica build delle calde + split probe tra rank

Misura: tempo skewed e traffico extra

Atteso: join scende, comunicazione sale

Dimostra: se/quando conviene rompere il floor.

6. Verifica determinismo skip-ahead

Come: confronta la concatenazione delle slice col generatore seq, byte a byte

Misura: checksum

Atteso: identici

Dimostra: che "niente scatter iniziale" non altera l'input.

Collegamento alla teoria

MPI – `Init_thread`/livelli di thread, `Alltoall`/`Alltoallv`/`Allreduce`, `Barrier` (L24–L26); **Hockney** $\alpha + \beta m$, comp/comm ratio (L23); load balancing distribuito (L14); Amdahl sulla collettiva.